

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.:T010	Name: Sagar Kandari
Class: TYIT	Batch:1
Date of Assignment:21-07-26	Date/Time of Submission:28-07-26

Part B - Program

a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Step 1 – Start.

Step 2 – Load WebForm1

. Step 3 – Check whether the page is loaded for the first time.

Step 4 – If No (PostBack):

Read the value stored in ViewState["count"].

Increment the value by 1.

Display the incremented value in Label2.

Step 5 – If Yes (First Load):

Store 1 in ViewState["count"].

Step 6 – When Button1 is clicked:

Read ViewState["count"].

Display its value in Label1.

Step 7 – When Button2 is clicked:

Create a cookie named "name".

Store the text entered in TextBox1 into the cookie.

Add the cookie to the response.

Redirect the user to WebForm2.aspx.

Step 8 – Stop.

Code:

ASPX.CX CODE-:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace practica_5c
{
    public partial class practical_5c : System.Web.UI.Page
```

```
{
    protected void Page_Load(object sender, EventArgs e)
    {
        if (IsPostBack)
        {
            int viewstateval = Convert.ToInt32(viewstate["count"]) + 1;
            Label2.Text = " view State = " + viewstateval.ToString();
        }
        else
        {
            viewstate["Count"] = 1;
        }
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        Label1.Text = viewstate["Count"].ToString();
    }
    protected void Button2_Click(object sender, EventArgs e)
    {
        HttpCookie h = new HttpCookie("name");
        h.Values = TextBox1.Text;
        Response.Cookies.Add(h);
        Response.Redirect("Webform1.aspx");
    }
}
}
```

WEBFROME2 CODE-:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace practical_5c
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if(Request.Cookies["name"]!=null)
            {
                Response.Write("welcome"+Request.Cookies["name"].Value);
            }
        }
    }
}
```

Output:

Label

Label

view state

sagar

cokies

Welcomesagar

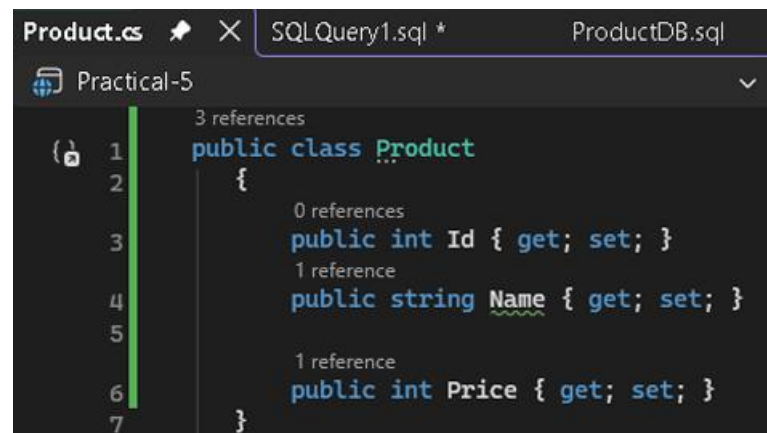
Label

b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm: 1. Start.

- 2. Create a database named ProductDB.**
- 3. Create the Product model with Id, Name, and Price.**
- 4. Create ApplicationDbContext and add the Products table (DbSet<Product>).**
- 5. Configure the SQL Server connection string in appsettings.json.**
- 6. Register ApplicationDbContext in Program.cs.**
- 7. Create HomeController and inject ApplicationDbContext.**
- 8. Create a new product object and assign values (e.g., Laptop, 50000).**
- 9. Add the product to the database using Add() and save it using SaveChanges().**
- 10. Install Entity Framework Core packages.**
- 11. Create and apply migrations using: o Add-Migration InitialCreate o Update-Database**
- 12. Run the application.**
- 13. Open Home/Index to insert the product.**
- 14. Verify the inserted record in SQL Server using SELECT * FROM Products.**
- 15. Stop.**

Code:



```
Product.cs  SQLQuery1.sql *  ProductDB.sql
Practical-5
1  public class Product
2  {
3      0 references
4      public int Id { get; set; }
5      1 reference
6      public string Name { get; set; }
7      1 reference
8      public int Price { get; set; }
9  }
```

Practical-5 HomeController

```
1 using Microsoft.AspNetCore.Mvc;
2 public class HomeController : Controller
3 {
4     private readonly ApplicationDbContext _context;
5     public HomeController(ApplicationDbContext context)
6     {
7         _context = context;
8     }
9     public IActionResult Index()
10    {
11        Product p = new Product();
12        p.Name = "Sachin's Laptop";
13        p.Price = 50000;
14        _context.Products.Add(p);
15        _context.SaveChanges();
16        return Content("Product Inserted Successfully");
17    }
18 }
```

Schema: <https://www.schemastore.org/appsettings.json>

```
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*",
9   "ConnectionStrings": {
10     "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True;"
11   }
12 }
13 }
```

Output:

Product Inserted Successfully

select * FROM Products;

T-SQL		Results		Message
	Id	Name	Price	
1	1	Laptop	50000	
2	2	Sagar's Laptop	50000	

- c) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Algorithm: 1. Start.

2. Open Visual Studio and create an ASP.NET Core MVC project.
3. Create the Employee model with required properties.
4. Create the ApplicationDbContext class.
5. Add DbSet<Employee> to the context.
6. Configure the connection string in appsettings.json.
7. Register ApplicationDbContext in Program.cs.
8. Install Entity Framework Core packages.
9. Create and apply database migrations.
10. Create the EmployeesController.
11. Implement the Create operation to add employee records.
12. Implement the Read operation to display employee records.
13. Implement the Edit operation to update employee records.
14. Implement the Delete operation to remove employee records.
15. Run the application.
16. Test Create, Read, Edit, and Delete operations.
17. Verify the output.
18. Stop.

Code:

EmployeesController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
using EmployeeCRUD.Data;

1 reference
public class EmployeesController : Controller
{
    private readonly ApplicationDbContext _context;

    0 references
    public EmployeesController(ApplicationDbContext context)
    {
        _context = context;
    }

    // GET: EMPLOYEES
    3 references
    public async Task<IActionResult> Index()
    {
        return View(await _context.Employees.ToListAsync());
    }

    // GET: EMPLOYEES/Details/5
    0 references
    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        Employee employee = await _context.Employees.FindAsync(id);

        if (employee == null)
        {
            return NotFound();
        }

        return View(employee);
    }

    // POST: EMPLOYEES
    0 references
    [HttpPost]
    public async Task<IActionResult> Create(Employee employee)
    {
        if (ModelState.IsValid)
        {
            _context.Add(employee);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        return View(employee);
    }

    // PUT: EMPLOYEES/5
    0 references
    [HttpPut]
    public async Task<IActionResult> Update(int id, Employee employee)
    {
        if (id != employee.Id)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
            _context.Update(employee);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        return View(employee);
    }

    // DELETE: EMPLOYEES/5
    0 references
    [HttpDelete]
    public async Task<IActionResult> Delete(int id)
    {
        if (id == null)
        {
            return NotFound();
        }

        Employee employee = await _context.Employees.FindAsync(id);

        if (employee == null)
        {
            return NotFound();
        }

        _context.Remove(employee);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
}
```

Employee.cs:

```

namespace EmployeeCRUD.Models
{
    13 references
    public class Employee
    {
        11 references
        public int ID { get; set; }
        12 references
        public required string Name { get; set; }
        12 references
        public required string Department { get; set; }
        12 references
        public double Salary { get; set; }
    }
}

```

AppDbContext.cs:

```

using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;

namespace EmployeeCRUD.Data
{
    7 references
    public class AppDbContext:DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options){}
        7 references
        public DbSet<Employee> Employees { get; set; }
    }
}

```

Appsetting.json:

```

{
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "MyConnection": "Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
    "AppDbContext": "Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}

```

Program.cs:

```

EmployeeCRUD Program
1 using Microsoft.EntityFrameworkCore;
2 using EmployeeCRUD.Data;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 // Add services to the container.
7 builder.Services.AddControllersWithViews();
8
9 builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));
10
11 var app = builder.Build();
12
13 // Configure the HTTP request pipeline.
14 if (!app.Environment.IsDevelopment())
15 {
16     app.UseExceptionHandler("/Home/Error");
17     // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
18     app.UseHsts();
19 }

```

In EmployeeCRUD.csproj add this :

```
<NuGetAuditSuppress Include="https://github.com/advisories/GHSA-g4vj-cjjj-v7hg" />
```

Output:WebApplication1 [Home](#) [Privacy](#)

Index

[Create New](#)

name	Department	salary	
sagar	it	10000000	Edit Details Delete

Delete

Are you sure you want to delete this?
WebApplication1.Models.Employee

name	sagar
Department	it
salary	10000000

[Delete](#) | [Back to List](#)

d) Authentication and Authorization in ASP.NET Core.**Algorithm:**

1. Create an ASP.NET Core MVC project.
2. Enable Cookie Authentication in Program.cs.
3. Create Login action in HomeController.
4. Verify username and password.
5. Redirect to Secure page after successful login.
6. Use [Authorize] to protect the Secure page.
7. Logout and redirect to Login.

Code:**Program.cs**

```
using Microsoft.AspNetCore.Authentication.Cookies;
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Home/Index";
    });
builder.Services.AddAuthorization();
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;

namespace AuthDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
```

```
{
    return View();
}

[HttpPost]
public async Task<IActionResult> Index(string username, string password)
{
    if (username == "admin" && password == "123")
    {
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, username)
        };

        var identity = new ClaimsIdentity(claims,
            CookieAuthenticationDefaults.AuthenticationScheme);

        await HttpContext.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationScheme,
            new ClaimsPrincipal(identity));

        return RedirectToAction("Secure");
    }

    ViewBag.Message = "Invalid Username or Password";
    return View();
}

[Authorize]
public IActionResult Secure()
{
    return Content("Welcome Sagar! Login Successful.");
}

public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(
        CookieAuthenticationDefaults.AuthenticationScheme);

    return RedirectToAction("Index");
}
}
```

```
<h2>Login</h2>
```

```
<form method="post">
```

Username :

```
<input type="text" name="username" /><br /><br />
```

Password :

```
<input type="password" name="password" /><br /><br />
```

```
<input type="submit" value="Login" />
```

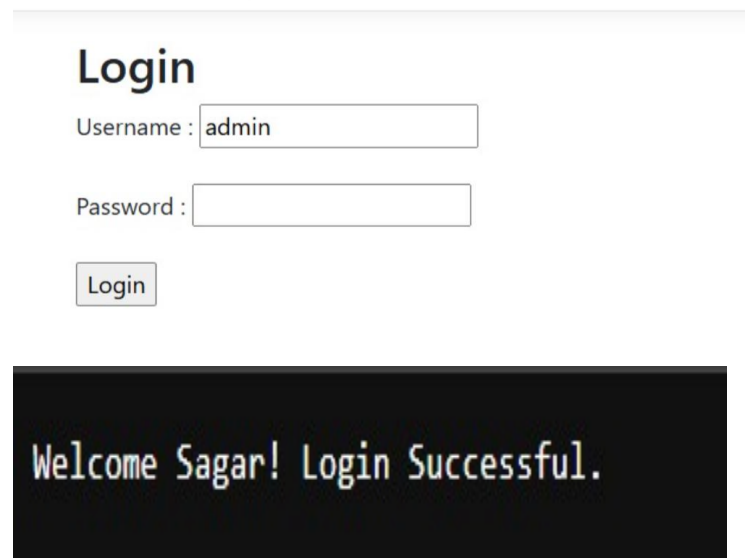
```
</form>
```

```
<p style="color:red">
```

```
@ViewBag.Message
```

```
</p>
```

Output:



Login

Username :

Password :

Welcome Sagar! Login Successful.